

Week 1 Functions in C++

Topics:

- Function declaration, definition & calling
- Return types & parameters
- Call by Value, Reference, and Pointer
- Function overloading

Functions

Functions in C++ are reusable blocks of code that perform a specific task. They allow modular programming, reduce redundancy, and increase readability. Functions can accept parameters, return values, and be overloaded to handle different data types.

Pass by value

Pass by value means that when a function is called, a copy of the actual argument is passed into the function's parameter. Any changes made inside the function only affect this copy, not the original variable. This approach ensures data safety, since the original variable remains unchanged, but it may be less efficient for large data types because copying takes extra memory and time. Pass by value is best suited for small, simple data where protection of the original data is important.

Pass by Reference

Pass by reference means that instead of passing a copy of the variable, the function receives a direct reference (alias) to the original variable. Any changes made inside the function affect the actual variable in the caller. This method is efficient since no copying is involved, and it allows the function to return multiple results by modifying multiple arguments. However, it also means the original data can be unintentionally changed if not handled carefully.

Pointer

A **pointer** is a variable that stores the **memory address** of another variable.

Declaration uses *. Example: `int *p;` declares a pointer to an integer.

To get the address of a variable, use **&**. Example: `p = &x;` assigns pointer p the address of x.

To access or change the value at that address, use * (dereferencing). Example: `*p = 20;` changes the value of x through the pointer.

Pointers allow efficient handling of arrays, dynamic memory, and functions.

Will be discussed in the next session in detail.

Pass by Pointer

Pass by pointer means that the function is given the memory address of a variable rather than the variable itself. Inside the function, the pointer can be dereferenced to access or modify the original value stored at that address. This approach allows functions to work with multiple outputs, dynamic memory, and large data structures efficiently. However, pointer passing requires careful handling to avoid errors like null pointer access or memory corruption.

C++ Examples with Code & Output

Increment Function (Call by Value)

Code:

```
#include <iostream>
using namespace std;
void increment(int a) { // 'a' is a copy of the argument
    a++;
    cout << "Inside function: " << a << endl;
}
int main() {
    int x = 5;
    increment(x);
    cout << "Outside function: " << x << endl;
    return 0;
}
```

Expected Output:

```
Inside Function = 6
Outside Function=5
```

Factorial Function (Call by Value)

Code:

```
#include <iostream>
using namespace std;
int factorial(int n) {
    if(n==0)
    {
        return 1;
    }
    else{
        return n * factorial(n-1);
    }
}
int main() {
    cout << "Factorial of 5 = " << factorial(5);
    return 0;
}
```

Expected Output:

Factorial of 5 = 120

Increment Function (Call by Reference)

```
#include <iostream>
using namespace std;
void increment(int &a) { // 'a' refers directly to the argument
    a++;
    cout << "Inside function: " << a << endl;
}
int main() {
    int x = 5;
    increment(x);
    cout << "Outside function: " << x << endl;
    return 0;
}
```

Expected Output:

Inside Function = 6
Outside Function=6

Swap using Reference

Code:

```
#include <iostream>
using namespace std;
void swapRef(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
int main() {
    int x=10, y=20;
    swapRef(x,y);
    cout << "x=" << x << " y=" << y;
    return 0;
}
```

Expected Output:

x=20 y=10

Pass by Pointer

```
#include <iostream>
using namespace std;
void increment(int *a)
{ // 'a' stores address of the argument
  (*a)++;
  cout << "Inside function: " << *a << endl;
}
int main() {
  int x = 5;
  increment(&x); // passing address of x
  cout << "Outside function: " << x << endl;
  return 0;
}
```

Expected Output:

Inside Function = 6

Outside Function=6

Swap using Pointer

Code:

```
#include <iostream>
using namespace std;
void swapPtr(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x=10, y=20;
    swapPtr(&x,&y);
    cout << "x=" << x << " y=" << y;
    return 0;
}
```

Expected Output:

x=20 y=10

Function Overloading

Code:

```
#include <iostream>
using namespace std;
int sum(int a, int b)
{
    return a+b;
}
double sum(double a, double b)
{
    return a+b;
}
int main() {
    cout << sum(3,4) << endl;
    cout << sum(2.5,3.7);
    return 0;
}
```

Expected Output:

7

6.2

1. Create a function to check if a number is prime using **pass by value**.
2. Create a function to reverse a number using **pass by reference**.
3. Create a function to check if a number is palindrome using **pointers**.
4. Create a recursive function to generate Fibonacci series using **pass by value**.
5. Create a function to find the maximum of three numbers using **pass by reference**.
6. Create a function to calculate the sum of digits of a number using **pointers**.
7. Create overloaded functions to calculate area of circle and rectangle using **pass by value**.
8. Create a function to count vowels in a string using **pass by reference**.
9. Create a recursive function to calculate power (x^n) using **pointers**.
10. Create a function to display multiplication table of a number using **pass by value**.
11. Create a function to swap three numbers using **pass by reference**.
12. Create a function to check Armstrong number using **pointers**.
13. Create a function to convert Celsius to Fahrenheit using **pass by value**.
14. Create a function to calculate factorial of a number using loop and **pass by reference**.
15. Create a function to check if a number is even or odd using **pointers**.
16. Create overloaded functions to calculate the area of a circle, rectangle, and triangle.
17. Create overloaded functions named sum that return the sum of two integers, two doubles, and two strings (concatenation).
18. Create overloaded functions named maxVal that return the maximum of two integers, three integers, and two floating-point numbers.